

PaperAgent: OPEA-based Cloud-Edge Academic Intelligence

Competition Technical Report (2 pages)

Problem	Academic writing increasingly relies on generative AI, but a cloud-only workflow may expose sensitive drafts and research materials, while a local-only workflow lacks scalable literature retrieval and service orchestration.
Solution	PaperAgent separates tasks according to privacy and compute characteristics: writing assistance runs locally on an AI PC, while literature retrieval and grounded QA are composed as modular OPEA services in the cloud.
AI for Good value	The system reduces privacy risk, lowers the barrier for academic assistance, and provides a reproducible open-source path for universities, researchers, and enterprise knowledge workers.

System Architecture

PaperAgent uses a coordinated cloud-edge layout. Privacy-sensitive grammar checking, academic polishing, and translation are executed locally through Qwen3-8B INT4 and HY-MT1.5-1.8B. Literature retrieval and grounded academic QA are implemented as OPEA MicroServices and composed by a MegaService.



Figure 1. PaperAgent cloud-edge architecture with edge AI services, OPEA cloud services, data layer, and end-to-end data flows.

Technical Implementation and Reproducibility

1. OPEA-based service composition

PaperAgent implements actual OPEA service composition rather than calling a single external API from a monolithic application. The cloud path includes a Retriever MicroService, a Prompt MicroService, an official OPEA LLM TextGen service, and a MegaService that exposes a unified endpoint. The effective runtime chain is: paperagent-retriever -> paperagent-prompt -> opea-service@llm -> PaperAgent MegaService.

2. Edge inference

The edge layer is not a mock interface. Qwen3-8B INT4 OpenVINO runs on Intel NPU for grammar checking and academic polishing. HY-MT1.5-1.8B is prepared in an OpenVINO-compatible INT4 path and runs locally for academic translation. This task placement keeps privacy-sensitive text processing on the AI PC.

3. Deployment workflow

- A single Windows entry point (deploy.bat) prepares the runtime environment, installs required dependencies, downloads models, and starts cloud and edge services.
- Model weights are not committed to the repository. The deployment flow downloads required runtime models automatically. ModelScope is the default source; Hugging Face is retained as a fallback.
- The public repository includes only sanitized synthetic demo data, safe configuration templates, health checks, and sensitive-source scanning scripts.

4. Evaluation focus

Dimension	What reviewers can verify
Functional integration	One-click deployment, model preparation, service startup, and unified UI entry.
Local AI capability	Grammar check / academic polish on local Qwen3 and translation on local HY-MT.
OPEA usage	Actual retriever, prompt, LLM, and MegaService composition; topology endpoint can be inspected.
Privacy-aware design	Sensitive writing assistance remains on the AI PC while literature retrieval and orchestration run in the cloud.
Open-source reproducibility	Apache-2.0 source code, deployment scripts, safe demo data, technical report, architecture figure, and CI validation.

5. Dependency note

The competition edition uses pypdf for local PDF page counting and fallback text extraction. MinerU remains available as the optional higher-fidelity parser when a runtime token is supplied. This keeps the fallback parser lightweight while avoiding redistribution concerns associated with the previous PDF dependency.

Conclusion

PaperAgent demonstrates a practical cloud-edge academic assistant: local AI inference is used where privacy matters, while OPEA services provide modular literature retrieval and grounded answer generation. The project is packaged as a reproducible, open-source competition submission with deployment automation and verifiable runtime components.